LAB 7: Fuzzing for Input Validation Bugs

ATM Withdrawal - Software Engineering Lab

Student	Abhishek Anand
SRN	PES1UG24CS017
Section	5A
Date	05 November 2026
Source	SETAPESU26/L6_L7_18_atm_withdrawal, Lab 7

Objective

Use Hypothesis property-based fuzzing to identify input-validation bugs in three ATM helper functions, harden those functions, and verify the fixes with regression tests.

Bugs identified and fixes applied

notes_needed() - Original code fails on nonnumeric/None inputs. Fixed code validates formatted nonnegative whole-number amounts and returns 0 for malformed input.

mask_card() - Original code assumes at least four space-delimited groups. Fixed code validates a 16-digit card, supports spaces/hyphens, and only reveals the last four digits.

average_withdrawal() - Original code fails on empty or malformed CSV input. Fixed code checks each amount and safely handles missing values before averaging.

Testing method

The original code and the fixed code were exercised independently with the same automated pytest/Hypothesis suite. Tests include fuzz-generated Unicode and missing inputs, valid-value properties, and explicit regression examples. The outputs shown below are captured from actual local execution.

Step 2 - Original buggy application

An invalid withdrawal amount causes the original script to terminate with an exception.

```
--- 01_buggy_application.txt (exit=1) ---
Enter an amount (e.g. 2,500): Traceback (most recent call last):
  File "C:\Users\anand\Downloads\Lab-7-automation-fixed\Software-Engineering-2026-\Lab-7\.lab7_runs\buggy\atm_withdrawal.py", line 29
, in <module>
    main()
  File "C:\Users\anand\Downloads\Lab-7-automation-fixed\Software-Engineering-2026-\Lab-7\.lab7_runs\buggy\atm_withdrawal.py", line 15
, in main
    result = notes_needed(user_input)
             ^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\anand\Downloads\Lab-7-automation-fixed\Software-Engineering-2026-\Lab-7\.lab7_runs\buggy\atm_withdrawal.py", line 2,
 in notes_needed
    return int(amount.replace(',', '')) // 100
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'abc'

Process exit code: 1
```

Evidence: `01_buggy_application.png`

Step 4 - Failing Hypothesis tests

Hypothesis exposes edge-case exceptions and invalid return values in the original implementation.

```
--- 02_failing_fuzz_tests.txt (exit=1) ---
FAILED test_atm_withdrawal.py::test_mask_card_invalid_returns_message[1234]
FAILED test_atm_withdrawal.py::test_mask_card_invalid_returns_message[1234 5678]
FAILED test_atm_withdrawal.py::test_mask_card_invalid_returns_message[0000 0000 0000 XXXX]
FAILED test_atm_withdrawal.py::test_mask_card_invalid_returns_message[None]
FAILED test_atm_withdrawal.py::test_average_invalid_returns_zero[] - ValueErr...
FAILED test_atm_withdrawal.py::test_average_invalid_returns_zero[100,,300] - ...
FAILED test_atm_withdrawal.py::test_average_invalid_returns_zero[100,hello]
FAILED test_atm_withdrawal.py::test_average_invalid_returns_zero[-100,200] - ...
FAILED test_atm_withdrawal.py::test_average_invalid_returns_zero[None] - Attr...
19 failed, 3 passed in 6.50s

Process exit code: 1
```

Evidence: `02_failing_fuzz_tests.png`

Step 5 - Fixed application

The same malformed values are handled without crashing; explicit safe defaults are returned.

```
--- 03_fixed_application.txt (exit=0) ---
Scripted console input: 'abc\n1234\n100,,300\n'
----------------------------------------------------------------
==== Running notes_needed ====
Enter an amount (e.g. 2,500): Number of 100 Notes: 0

==== Running mask_card ====
Enter a card number (e.g. 1234 5678 9012 3456): Masked Card: Invalid card number

==== Running average_withdrawal ====
Enter withdrawals as CSV (e.g. 500,1000,2000): Average Withdrawal: 0

Process exit code: 0
```

Evidence: `03_fixed_application.png`

Step 6 - Passing Hypothesis tests

Property-based and regression tests pass against the hardened implementation.

```
--- 04_passing_fuzz_tests.txt (exit=0) ---
PASSED test_atm_withdrawal.py::test_mask_card_invalid_returns_message[1234]
PASSED test_atm_withdrawal.py::test_mask_card_invalid_returns_message[1234 5678]
PASSED test_atm_withdrawal.py::test_mask_card_invalid_returns_message[0000 0000 0000 XXXX]
PASSED test_atm_withdrawal.py::test_mask_card_invalid_returns_message[None]
PASSED test_atm_withdrawal.py::test_average_invalid_returns_zero[]
PASSED test_atm_withdrawal.py::test_average_invalid_returns_zero[100,,300]
PASSED test_atm_withdrawal.py::test_average_invalid_returns_zero[100,hello]
PASSED test_atm_withdrawal.py::test_average_invalid_returns_zero[-100,200]
PASSED test_atm_withdrawal.py::test_average_invalid_returns_zero[None]
22 passed in 2.78s

Process exit code: 0
```

Evidence: `04_passing_fuzz_tests.png`

Result and conclusion

The original implementation's input assumptions were demonstrated to fail on edge cases. Defensive validation was added to all three functions, and the same suite was used to check that the corrected code no longer crashes and preserves expected behavior on valid inputs.

Submitted source files: **atm_withdrawal.py** and **test_atm_withdrawal.py**.